

■ BANK ACCOUNT

From a real-world banking problem → to a Python-based solution

■ THE BUSINESS PROBLEM

A simple banking system still needs to manage customer accounts, balances, deposits

The challenge:

How can I build a system that manages these operations while keeping records saved

■ WHAT WAS I TRYING TO BUILD?

A menu-driven banking system that allows users to:

- 1 Create an account
- 2 Deposit money
- 3 Withdraw money
- 4 Check balance
- 5 View transaction history
- 6 Exit the system

Key requirement: the records had to persist.

Goal: turn individual Python concepts into one working application

■ MY APPROACH

I broke the problem into smaller functions instead of writing one large program.

Create Account → Deposit → Withdraw



Check Balance



Transaction History



Save Records

Modular functions made the system easier to build, test and understand.

■■ THE SOLUTION

- Python — application logic & user interaction
- Lists & dictionaries — structured account data
- JSON — persistent storage
- datetime — transaction timestamps
- `save_records()` — writes updated records to `bank.json`

User Action → Function → Update → Transaction → Save

The solution connects program logic with persistent data storage.

■ HOW THE DATA IS STRUCTURED

Each account stores:

- Account Number
- Account Holder Name
- Balance
- Transactions
 - Type
 - Amount
 - Date & Time

This keeps account information and its transaction history together.

Example: `account_number • name • balance • transactions`

■ BUSINESS RULES & TESTING

I tested the main banking scenarios:

- Duplicate account number → stopped
- Deposit → balance updated + transaction recorded
- Withdrawal → balance checked first
- Withdrawal above balance → “Insufficient balance!”
- Invalid account → “Account not found!”
- Transaction history → type, amount & date/time displayed

Build it → test it → find issues → improve the logic

■ THE FINAL SYSTEM

A menu-driven interface connects all the functions:

1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Transaction History
6. Exit

The system was tested end-to-end and records were saved to JSON.

From separate functions → to a complete working system

■ WHAT I LEARNED

This project helped me move beyond individual Python programs and think about building

Technical learning:

- Functions & modular programming
- Lists & dictionaries
- Loops & conditional logic
- JSON & file handling
- Data persistence
- Date/time handling
- Debugging & testing

Biggest takeaway: solve the real-world problem first, then break it into manageable pieces.

■ WHAT'S NEXT?

If I were to scale this system further:

- PIN/password authentication
- Account search
- Account-to-account transfers
- Mini statements
- Transaction IDs
- Stronger input validation
- SQL/database integration
- Graphical user interface

JSON → Database

CLI → Application

This project gives me a foundation for the next level of development.

■ PROJECT 03 • BLEEP INDIA

BANK ACCOUNT MANAGEMENT SYSTEM

Python | JSON | File Handling | Functions

Built to practice:

Problem Solving • Python • Data Handling • Application Logic

■ GitHub: Add your repository link